

6. 配置管理

6.1 课程工具标识

工具标注：本部分使用了以下课程介绍的工具完成配置管理。

- **Git** (版本控制)
- **GitHub** (远程仓库+CI/CD)
- **GitHub Actions** (持续集成与持续部署)
- **pubspec.lock** (依赖锁定)
- **Supabase Migrations** (数据库配置管理)

6.2 配置管理目标

配置管理共有四个目标，分别是：一、可追溯，每一次功能变更、文档修改、测试结果都有明确的文件记录和提交记录；二、可复现，通过锁定依赖 (**pubspec.lock**) 和标准化命令，保证项目可在任意环境下重新构建和测试；三、可交付，明确交付包范围，避免漏交源码、报告、UML 文件或配置文件；四、可回退，基于 **Git** 历史记录，支持版本回退和问题定位。

6.3 配置项清单

配置项	路径	类型	管理方式
Flutter 源码	lib/	源代码	Git 跟踪
自动化测试代码	test/	测试代码	Git 跟踪
项目依赖声明	pubspec.yaml	配置文件	Git 跟踪
依赖版本锁定	pubspec.lock	锁文件	Git 跟踪
代码规范配置	analysis_options.yaml	配置文件	Git 跟踪
CI/CD 工作流	.github/workflows/flutter-ci-cd.yml	自动化配置	Git 跟踪
数据库结构/迁移	supabase/migrations/	SQL 脚本	Git 跟踪
种子数据	supabase/seed	SQL 脚本	Git 跟踪
各平台工程文件	android/,ios/,web/,linux/,macos/,windows/	工程配置	Git 跟踪
项目报告与文档	docs/	文档	Git 跟踪
UML 源文件	docs/uml/	设计文件	Git 跟踪
项目 README	README.md	文档	Git 跟踪

6.4 不纳入版本控制的内容

内容	原因
build/	编译产物，可随时重新生成
.dart_tool/	本地工具缓存，与环境相关
IDE 配置文件 (如.vscode/,.idea/)	与个人开发环境相关
SUPABASE_URL、SUPABASE_ANON_KEY	敏感信息，通过环境变量或 GitHub Secrets 注入
真实支付证书/密钥	安全风险，不提交到仓库

6.5 版本控制

6.5.1 分支管理

分支	用途
main	课程大作业主线和最终提交基线
feature/*	功能开发分支 (如 feature/user-booking)

docs/*	文档整理分支
fix/*	缺陷修复分支

本项目采用小团队简化流程: 主要在 **main** 分支进行小步提交, 确保每次提交后 **flutter test** 通过。

6.5.2 版本命名规则

版本	对应功能
v0.1.0	Flutter 工程初始化+三端入口骨架
v0.2.0	普通用户端: 预约、取消、查看
v0.3.0	司机端: 任务、统计、位置共享
v0.4.0	后台: 车辆/司机管理、实时监控
v0.5.0	调度、支付、信用、通知、数据分析

6.5.3 提交信息规范

类型	说明	示例
feat	新功能	feat: 用户端预约车次功能
fix	缺陷修复	fix: 司机端位置共享断连问题
docs	文档	docs: 补充配置管理章节
test	测试	test: 增加取消预约扣分测试
refactor	重构	refactor: 优化调度状态管理
chore	构建/配置	chore: 升级 Flutter 依赖

6.6 变更控制流程

为规范项目变更, 确保每次修改都有记录、可追溯、可回退, 制定如下变更控制流程:

第一步, 提出变更。任何团队成员发现问题或提出新需求时, 可通过口头沟通、即时消息或会议记录方式提出变更申请。

第二步, 判断影响范围。由项目负责人或技术负责人评估本次变更的影响范围, 包括: 是否涉及核心功能 (预约、支付、调度、位置共享等), 是否需要修改需求文档或设计文档, 是否影响已有测试用例。

第三步, 分支判断与操作。若影响核心功能, 必须先更新对应的需求文档 (**docs/02_项目需求.md**) 或设计文档 (**docs/03_项目设计.md**), 并在文档中标注变更内容、变更原因和变更日期。然后再进行代码或配置的修改。若不涉及核心功能, 可以直接修改源码、配置文件或文档, 无需提前更新需求/设计文档。

第四步, 实现变更。在相应的开发分支 (如 **feature**、**fix**、**docs**) 上完成代码或文档修改。若项目规模较小, 也可以在 **main** 分支上直接提交。

第五步, 执行自动化测试。本地运行 **flutter test**, 确保所有已有测试用例通过。若新增功能, 需同步补充对应的测试用例。

第六步, 测绘结果判断。若测试通过, 将变更提交到本地仓库, 并推动到远程仓库。提交信息需遵循第 6.5.3 节规定的提交规范 (如 **feat: 用户端预约车次功能**)。若测试不通过, 根据测试报告定位缺陷, 修复问题后返回第 4 步重新实现变更, 直到所有测试通过为止。

第七步, 形成新基线。变更合入 **main** 分支后, 更新相应的基线版本 (参考第 6.7 节基线管理), 并在项目报告中记录本次变更的简要说明。

6.7 基线管理

基线	包含内容	验收条件
需求基线	docs/02_项目需求.md+docs/uml/use_case_diagram.puml	功能需求、业务规则、验收标准完整
设计基线	docs/03_项目设计.md+docs/uml/*.puml	架构、模块、数据模型、核心流程完整
源码基线	lib/+test/+pubspec.yaml	三端功能可运行，测试通过
测试基线	docs/05_项目测试.md+test/	关键主流程+业务规则自动化测试通过
交付基线	README+docs+源码+测绘+配置	满足课程提交清单要求

6.8 CI/CD 配置管理

工具标注：本部分使用 GitHub Actions 完成持续集成与持续部署

项目	配置内容
工作流文件	.github/workflows/flutter-ci-cd.yml
触发条件	推送到 main、指向 main 的 PR、手动触发
CI 步骤	安装依赖->格式检查->静态分析->自动化测试->Web 构建
CD 步骤	main 分支验证通过后，将后台管理端 Web 部署到 GitHub Pages
部署入口	后台管理端 (--dart-define=APP_MODE=admin --dart-define=BACKEND=mock)
Pages 配置要求	仓库 Settings > Pages > Build and deployment 选择 GitHub Actions

6.9 依赖与后端配置管理

6.9.1 Flutter 依赖管理

使用 pubspec.yaml 声明依赖，使用 pubspec.lock 锁定精确版本，安装命令为 flutter pub get

6.9.2 环境配置管理

通过 --dart-define 实现多环境配置：

环境变量	可选值	说明
APP_MODE	passenger/driver/admin	三端入口切换
BACKEND	auto/mock/supabase	后端模式切换

6.9.3 Supabase 配置管理

配置项	管理方式
数据库结构	supabase/migrations/下的 SQL 迁移脚本
演示数据	supabase/seed/下的 SQL 种子文件
敏感信息	通过 --dart-define 或 GitHub Secrets 注入，不提交到仓库
CI 默认策略	使用 BACKEND=mock，避免公开仓库依赖私有 Supabase

6.10 构建与验证命令

命令	用途
flutter pub get	安装依赖

dart format lib test	格式化代码
flutter analyze	静态分析
flutter test	执行自动化测试
flutter build web --release --dart-define=APP_MODE=admin	构建后台管理端 Web
flutter run --dart-define=APP_MODE=passenger	启动普通用户端
flutter run --dart-define=APP_MODE=driver	启动司机端
flutter run -d chrome --dart-define=APP_MODE=admin	启动后台管理端

6.11 交付包结构

最终提交的目录结构如下：

school_bus_app/

├── lib/	# Flutter 源码
├── test/	# 自动化测试
├── docs/	# 项目报告 + UML + 文档
├── android/	# Android 平台工程
├── ios/	# iOS 平台工程
├── web/	# Web 平台工程
├── linux/	# Linux 桌面工程
├── macos/	# macOS 桌面工程
├── windows/	# Windows 桌面工程
├── supabase/	# 数据库迁移与种子数据
├── .github/workflows/	# CI/CD 配置
├── pubspec.yaml	# 依赖声明
├── pubspec.lock	# 依赖锁定
├── analysis_options.yaml	# 代码规范
├── README.md	# 项目说明
└── 校车管理系统_技术方案.md	# 技术方案文档

6.12 配置管理总结

管理维度	使用工具/方法	关键文件/位置
版本控制	Git + GitHub	本地仓库+远程仓库
分支策略	Git Flow 简化模型	.git 配置
依赖管理	pubspec.yaml + lock	项目根目录
环境配置	--dart-define	启动命令/CI
敏感信息	GitHub Secrets	仓库 Settings
CI/CD	GitHub Actions	.github/workflows
数据库迁移	Supabase Migrations	supabase/migrations/
代码规范	analysis_options.yaml	项目根目录
文档管理	Markdown+Git	docs/目录